Security Features in Real-World Applications

1. Identify any five real-world applications or online services that involve communication, authentication, or transactions.

For this assignment, I picked apps I actually use every day, so I could reason about why each security feature matters rather than just listing textbook definitions.

1. WhatsApp (personal chats)

The biggest worry with any messaging app is someone reading your conversation while it's in transit, a classic **confidentiality** problem. WhatsApp handles this with end-to-end encryption, built on the Signal Protocol. What's interesting here is that the key exchange (X3DH) happens once when a chat is started, and then every single message gets a *new* encryption key through something called the Double Ratchet so even if one message's key is somehow leaked, past and future messages stay safe. That's a much stronger guarantee than "the connection is encrypted," which is what most people assume.

2. IRCTC (train ticket booking)

Anyone who's tried booking a Tatkal ticket knows this site gets hammered with traffic at 10 AM sharp. So the feature that actually matters most here isn't fancy encryption, it's **availability**. IRCTC uses load balancing and traffic queuing (virtual waiting rooms) to keep the site from collapsing under simultaneous requests. Underneath, they also run HTTPS/TLS for the payment part, but the thing that decides whether you even get to see the payment page is the queue system managing server load.

3. Google Pay / UPI

UPI transactions are a good example of layering two *different* protections. First, the app itself needs to know it's really you that's **authentication**, done through your phone's biometric lock or app PIN. But even after you're "in," transferring money needs a second, separate check of the UPI PIN which is **authorization**: proving you're allowed to approve *this specific* transaction. Underneath both, TLS and bank-side PKI (digital certificates) protect the data as it moves between your phone, the UPI switch (NPCI), and the bank server.

4. Zoom (video calls)

During COVID, Zoom actually had a scandal: people randomly joining meetings and disrupting them ("Zoom-bombing"). The gap was **authorization**: anyone with a link could join. Their fix was meeting passwords + waiting rooms, controlled through AES-256 encryption for media streams, which also gave a confidentiality boost as a side effect. It's a

decent case study because it shows a security feature being *added after* an attack rather than designed in from day one.

5. Amazon (order checkout)

The thing nobody thinks about here is **integrity** making sure the price/item you agreed to isn't silently changed between clicking "Buy" and the order actually processing. Amazon uses hashing (SHA-256/HMAC) on transaction requests so that if even one digit of the order amount is tampered with mid-transit, the hash won't match on the server side and the request gets rejected.

Application	**Feature**	**Security Mechanism**	**Protocol / Technology**	**How it provides security**
WhatsApp	Confidentiality	End-to-end encryption, per-message key rotation	Signal Protocol (X3DH + Double Ratchet)	Only sender and receiver devices hold the decryption keys; keys change per message so a single leak doesn't expose the whole conversation
IRCTC	Availability	Load balancing, virtual queuing	Traffic management + HTTPS/TLS for payments	Distributes/queues massive simultaneous requests so the service doesn't crash during peak booking windows
Google Pay (UPI)	Authorization	UPI PIN verification	UPI protocol (NPCI) + PKI/TLS	Requires a separate PIN beyond login to approve a specific fund transfer, stopping misuse even if the app session is already unlocked
Zoom	Authorization / Confidentiality	Meeting password, waiting room, media encryption	AES-256	Prevents uninvited users from joining and encrypts video/audio streams so intercepted traffic is unreadable

Amazon	Integrity	Hashing of transaction data	SHA-256 / HMAC	Detects if order details were altered in transit by comparing computed hash values before processing payment

2. For any five real-world applications, investigate the security controls used to protect users and data. Classify the controls based on whether they provide Confidentiality, Integrity, or Availability, and determine how these controls help in preventing, detecting, and recovering from security attacks. Mention the mechanisms and protocols involved.

1. WhatsApp

The core control here is end-to-end encryption (Signal Protocol), which is a **Confidentiality** control — but it quietly does an integrity job too. If an attacker tampers with an encrypted message in transit, the decryption on the receiver's end simply fails/garbles, which is how tampering gets detected almost as a side effect of the encryption design, not a separate check.

- **Prevent:** Per-message key rotation (Double Ratchet) means even if one key is compromised, the attacker can't decrypt earlier or later messages; this limits how much damage a single break-in causes, rather than stopping the break-in itself.
- **Detect:** WhatsApp shows a "security code changed" notification if a contact's encryption key suddenly changes; this is the app's way of flagging a possible man-in-the-middle attempt.
- **Recover:** There isn't really a "recovery" for a leaked message (it's already read), which is actually a good discussion point; some attacks are only preventable, not recoverable, and that's a real limitation of E2E systems.

2. IRCTC

The dominant control is traffic management (load balancing + queuing), which is an **Availability** control, since the whole point is keeping the site reachable during a Denial-of-Service-like surge of real users (or sometimes actual bot-driven DoS attempts).

- **Prevent:** Rate limiting and virtual queues stop the server from being overwhelmed in the first place by spacing out how many requests hit the booking engine per second.
- **Detect:** Anomaly detection on traffic patterns (e.g., unusually high requests from a single IP) flags likely bot activity versus genuine ticket-seekers.

- **Recover:** Auto-scaling of server instances during peak load, and database transaction rollback if a booking fails mid-process, so a half-completed transaction doesn't corrupt seat inventory data.

3. Google Pay / UPI

This one's a nice example of two controls working together for two different CIA properties. The UPI PIN is an **Integrity/Authorization** control (only the rightful owner can approve the exact transaction amount), while TLS encryption of the data channel is a **Confidentiality** control.

- **Prevent:** The PIN stops unauthorized transactions even if someone gets hold of an unlocked phone; TLS stops attackers from reading transaction data over public Wi-Fi.
- **Detect:** Banks run real-time fraud-detection engines that flag transactions inconsistent with a user's usual pattern (e.g., a sudden large transfer at 3 AM to a new account).
- **Recover:** The UPI dispute/reversal mechanism through NPCI allows a wrongly-processed or fraudulent transaction to be reversed within a set window this is the recovery layer most people don't know exists until they need it.

4. Zoom

Waiting rooms and meeting passwords are **Authorization** controls, while AES-256 encrypted media streams are a **Confidentiality** control. This is worth noting because Zoom's 2020 issues weren't really an encryption failure; the encryption was fine, the authorization layer was just too weak (guessable meeting IDs, no password by default).

- **Prevent:** Waiting rooms stop uninvited participants from even joining, addressing the actual root cause of Zoom-bombing rather than the symptom.
- **Detect:** Host-side alerts show when someone tries to rejoin after being removed, letting the host know an attack attempt is ongoing.
- **Recover:** Hosts can lock a meeting mid-session and remove a disruptive participant, cutting off further damage without ending the whole call.

5. Amazon

The main control is hashing/HMAC validation on transaction data, which is an **Integrity** control, backed by HTTPS/TLS for **Confidentiality** and a Web Application Firewall (WAF) for **Availability** against bot traffic and injection attacks.

- **Prevent:** The WAF blocks known attack patterns (SQL injection, XSS) at the network edge before they ever reach Amazon's application servers.
- **Detect:** Server-side logging and monitoring compare incoming request hashes against expected values, catching tampered orders before payment is charged.

- **Recover:** If a corrupted or fraudulent order does slip through, Amazon's order-management system can roll it back using transaction logs and issue a refund, restoring the correct account/inventory state.

Application	Main Control	CIA Property	Prevent	Detect	Recover
WhatsApp	End-to-end encryption (Signal Protocol)	Confidentiality	Per-message key rotation limits blast radius	"Security code changed" alert	Not recoverable once read — a real limitation
IRCTC	Load balancing / queuing	Availability	Rate limiting prevents server overload	Traffic anomaly detection flags bots	Auto-scaling + transaction rollback
Google Pay (UPI)	PIN + TLS	Integrity/Authorization + Confidentiality	PIN blocks unauthorized approval	Bank fraud-detection engines	UPI dispute/reversal process
Zoom	Waiting room + AES-256	Authorization + Confidentiality	Blocks uninvited join attempts	Rejoin alerts after removal	Host can lock/remove mid-session
Amazon	Hashing/HMAC + WAF	Integrity + Availability	WAF blocks injection attacks	Hash mismatch flags tampering	Order rollback + refund

One thing I noticed doing this: almost none of these apps rely on a single control for a single property: Confidentiality, Integrity, and Availability controls are usually stacked together, and a weakness in one (like Zoom's authorization gap) can undermine an otherwise-strong system even when the encryption itself is solid. That's really the practical takeaway : CIA isn't three separate boxes to tick, it's a set of controls that have to work together.